Christ Link Social — Native App Path to Launch

Oct 18, 2026 · @David

Where you are

You're much further along than the old plan assumed: ChristLink-native is a real Expo app, not a Median wrapper, and nearly every feature from the build plan's Phases 0–5 is written. The big change in this plan is the backend: Christ Link moves to its own NestJS API on Railway with Postgres and Redis, and all files move to Cloudflare R2, the same setup as Vendly, replacing both server.js and Supabase. That adds about 8 weeks, so the full path to launch is about 17 weeks part-time.

This plan replaces the Median steps in the earlier Christ Link build plan; the Median wrapper isn't needed anymore.

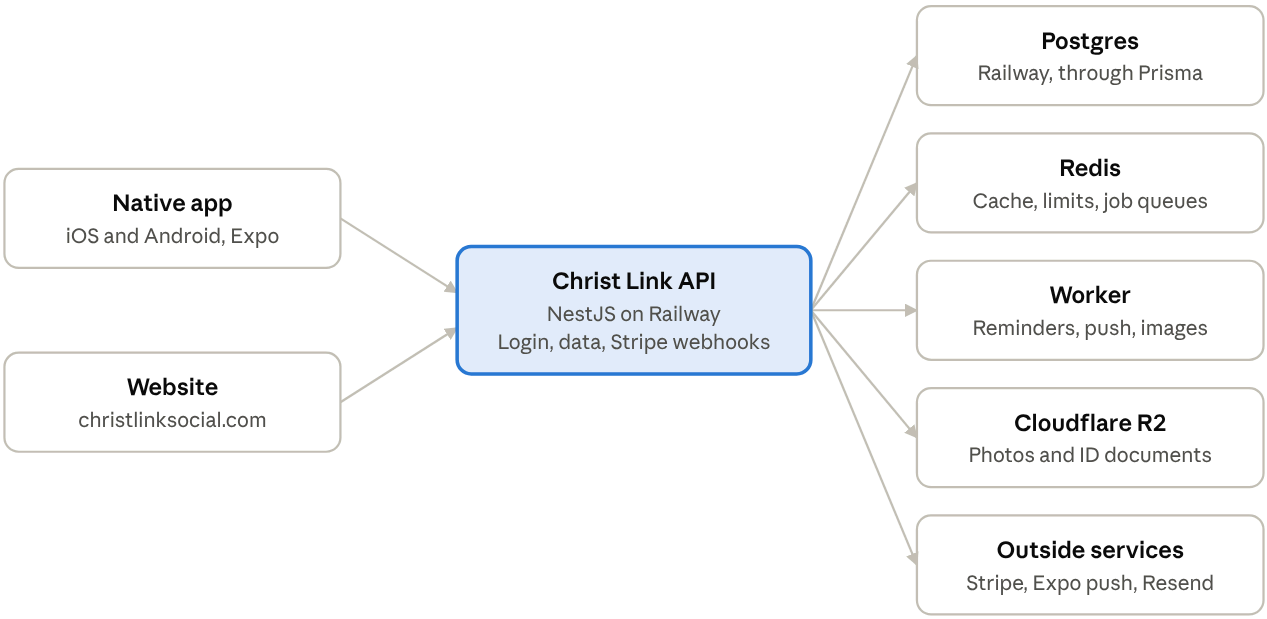
Area	State in the repo	What's left
Screens and features	About 35 screens and 120 components: events, RSVP, paid tickets with Apple Pay, QR tickets, check-in scanner, host tools, community, Prayer Wall, Church Pages, Serve, friends going, verse and share cards, admin, settings	Run them on a phone and fix what breaks
Design system	Light and dark tokens, components, /styleguide, floating tab bar, signature moments	Motion uses React Native's older animation system in 15 files; fine-tune in Phase E
Tests and CI	154 unit tests, including an API contract test listing every route the app calls; CI type-checks, lints and bundles iOS and Android	Keep green; the contract test becomes the checklist for the new API

Area	State in the repo	What's left
Data today	The ChristLink1 Express API (about 100 routes) plus Supabase for login, storage and the new features	Replaced by the Railway backend in Phases B–D
Railway backend (backend/ folder)	NestJS + Prisma + Redis scaffold: health check and a User table only	Build it out to full parity (Phases B and C)
Expo SDK 51	Too old for Apple's current upload rules and for today's Expo Go	Upgrade (next section)
Stripe key	.env.local uses a live key	Switch to a test key now
Launch setup	Store listings, privacy labels and legal drafts written in docs/STORE.md and content/legal.ts	Accounts, demo reviewer login, lawyer review, live keys

The backend and two rules

1. One Railway backend, files on R2 (decided). The native app and the website both talk to a single NestJS API on Railway. Postgres holds all data, Redis handles caching, rate limits and background job queues, a worker service on Railway runs reminders, pushes, image resizing and emails, and Cloudflare R2 stores every file. This matches Vendly, so fixes and patterns carry over between the two apps, and nothing depends on Supabase anymore.

One Railway API serves the app and website; files live on R2



Christ Link backend after the switch · Railway + Cloudflare R2

Photos upload straight from the phone to R2 through short-lived upload links the API hands out, so large files never pass through the API itself.

Two rules make the switch safe:

- **Same API shape.** The new API keeps the existing /api routes and response fields from server.js wherever possible, so the website only needs its login and upload code changed, and the native app's API contract test becomes the checklist for what to build
- **Build alongside, then switch once.** The current site keeps running on server.js and Supabase until the new backend passes every test with copied data. Then one planned switch moves data, files and both apps over together

Users won't need new accounts: sign-in is by email code, so after the switch they just enter their email again.

2. Upgrade Expo from SDK 51 before anything else. Since April 28, 2026, Apple only accepts apps built with Xcode 26, and Expo recommends at least SDK 54 for that. The App Store version of Expo Go also only runs recent SDKs, so SDK 51 can't be tested on your phone with Expo Go. Upgrading also unlocks Reanimated 4, which Emil Kowalski's

animation skills are written for (Phase C). Target the newest SDK that Expo Go on the App Store supports when you start (SDK 54 or later), one SDK version at a time, testing between each.

Phase A: Get it running on a real phone (weeks 1–2)

Goal: the app runs on your iPhone and an Android phone against a development database, with every flow tested by hand. Same working rules as before: one step per session, a branch per step, test before merging, and end each session by updating docs/PROGRESS.md.

Step A.1 — Clean up and switch to test mode

```
Make sure .env.local is in .gitignore and has never been committed; if it
was, tell me which keys to rotate. Replace the live Stripe publishable key in
.env.example and in my instructions with a pk_test_ placeholder, and add a
startup warning in development builds if a pk_live_ key is detected. Update
CLAUDE.md to say the backend is moving to backend/ (NestJS on Railway with
Postgres, Redis and Cloudflare R2), that server.js and Supabase stay in use
until the switch in Phase D of docs/LAUNCH.pdf, and that no new features go
into server.js or Supabase from now on.
```

Done when:

- [ ] Your local .env.local uses a pk_test_ key and CLAUDE.md describes the backend plan

Step A.2 — Upgrade the Expo SDK

```
Upgrade this app from Expo SDK 51 to the newest SDK that Expo Go on the App
Store currently supports (at least SDK 54), one SDK version at a time,
following Expo's upgrade guides for each version. Use npx expo install --fix
for dependency versions, move to the New Architecture, upgrade react-native-
reanimated to version 4 with react-native-worklets, and update Sentry,
Stripe, maps and any other native libraries to their compatible versions.
After each SDK step run typecheck, lint, tests and both bundles, and fix what
breaks before the next step. Update eas.json to an Xcode 26 build image.
Record every breaking change you fixed in docs/PROGRESS.md.
```

Done when:

- [ ] The app opens in the App Store version of Expo Go on your phone
- [ ] CI passes on the new SDK

Step A.3 — Development build and first test of what works today

```
Create an EAS development build for iOS and Android (needed for Apple Pay and remote push, which Expo Go can't do) and walk me through installing it on my phones. Then guide me through test flows 1–6 in docs/PROGRESS.md against the current API in test mode, one at a time. After each flow, ask me what broke or felt wrong, fix screen and app-side bugs, and log anything that's a server problem in docs/BACKEND-NOTES.md for the new backend instead of fixing server.js. Skip the Prayer Wall, churches, Serve and settings flow; those get tested on the new backend in Phase D.
```

Done when:

- [ ] Flows 1–6 work on a real iPhone and a real Android phone, or their server issues are logged
- [ ] A test payment with card 4242 4242 4242 4242 and with Apple Pay creates a ticket

Phase B: Finish the server side (week 3)

Goal: close the gaps the native app documented. These changes go in the ChristLink1 repo (server.js), so open that folder in Claude Code, not the native one. Remember Railway deploys ChristLink1's main automatically, so test on a branch first.

Step B.1 — Server fixes for the native app

```
The native app in NolimitDae/ChristLink-native documents server gaps in its CLAUDE.md, docs/STATUS.md, docs/SECURITY.md and docs/deeplinks/README.md. Read those files from that repo, then fix them here in server.js and schema.sql, on a branch: 1. Add a signed GET /api/tickets/:id/wallet-link endpoint for Apple Wallet passes. 2. Make GET /api/events/:id/attendees honor the user_settings privacy flag that hides RSVPs from non-friends. 3. Fix /api/price-breakdown returning negative host earnings when price=0. 4. Add the missing ticket-type update endpoint. 5. Serve /.well-known/apple-app-site-association and /.well-known/assetlinks.json from the files in docs/deeplinks, and redirect /event/:id to /#event/:id. 6. Scope the older tables' row-level security policies to service_role as SECURITY.md describes. 7. Insert rows into the notifications table for ticket confirmations, new events from followed hosts and churches, forum replies, refund updates and verification results, so the push-dispatch function sends them. Add tests, and keep the web app working exactly as before.
```

Done when:

- [ ] Tapping a shared christlinksocial.com/event link on your phone opens the native app
- [ ] Buying a test ticket sends a push confirmation, and the Wallet button adds a pass
- [ ] The website still works the same in a browser

Phase C: Fine-tune the UI with Emil Kowalski's skills (weeks 4–6)

Goal: the app feels native and polished on a mid-range phone, not just in screenshots. Emil Kowalski's skills (from his work at Vercel and Linear) teach Claude Code the small motion and design decisions that agents usually get wrong. Do this after Phase A, because the skills assume Reanimated 4.

Install the skills into the repo (once, from the ChristLink-native folder):

```
npx skills@latest add emilkowalski/skills
```

Which skills apply to this app

Skill	Use it for	How it runs
animate-expo	Building and rebuilding every animation in this Expo app: press feedback, sheets, gestures, haptics, screen transitions	Automatically when Claude animates something
improve-animations	A full audit of the app's motion with prioritized fix plans	Automatically when you ask for an audit
find-animation-opportunities	Finding the few places that should animate and rejecting the rest	Automatically
apple-design	Springs, sheets, materials, typography and restraint, Apple's way	Automatically
emil-design-eng	General polish: shadows, borders, spacing, the invisible details	Automatically

Skill	Use it for	How it runs
break-ui	Stress-testing screens with long names, huge counts, emoji and empty lists	Automatically
review-animations	A strict review of motion changes before you merge	Only when you type /review-animations
prototype	Building 2–4 versions of one piece of UI to compare side by side	Only when you type /prototype

Skip mobile-native and animation-performance (written for websites), plus ask-sonner, write-swift and pick-ui-library, which don't fit this app.

What I already spotted (from reading the code; the audit in C.1 will be more complete):

- 15 components animate with React Native's built-in Animated library and none use Reanimated, even though it's installed. animate-expo's rule is Reanimated only, so motion runs on the UI thread and doesn't stutter while the app loads data
- The I-prayed count (RollingNumber) animates on the JavaScript thread, which drops frames when the Prayer Wall is busy
- The RSVP celebration's ticket landing uses an ease-in curve; arriving motion should ease out
- Every screen push slides from the right, including modals reached often; the skill reserves animation for occasional moments and says tab switches never slide
- The theme's spring settings (damping 18, stiffness 220 from the design doc) should move to the duration and damping-ratio form the skill specifies

Step C.1 — Motion audit (read-only)

```
Use the improve-animations skill to audit every animation and transition in
this app: components/ui, components/community, components/event,
components/home, components/tickets, the TabBar and the stack options in
app/_layout.tsx. Judge each against the animate-expo rules and
docs/DESIGN.pdf's signature moments. Write docs/MOTION-AUDIT.md with findings
ranked by impact and a self-contained plan for each fix. Don't change any
code.
```

Done when:

- [ ] You've read MOTION-AUDIT.md and crossed out anything you disagree with

Step C.2 — Rebuild motion on Reanimated (one group per session)

Run this prompt once for each group, in order: (1) Button, cards and chips press feedback; (2) BottomSheet and the checkout sheet; (3) PrayerCard glow and RollingNumber; (4) LightBurst and RsvpCelebration; (5) EventHero stretchy header; (6) Skeleton shimmer, Badge pulse and Segmented.

```
Using the animate-expo skill and plan [number] in docs/MOTION-AUDIT.md, rebuild the animations in [group] with Reanimated 4: CSS transitions for simple state changes, shared values only where a finger or scroll drives the motion, spring configs from the skill's table, and haptics on the signature moments. Ship the Reduce Motion version in the same change. Keep the look in docs/DESIGN.pdf. Then run /review-animations on your own diff and fix everything it flags before you finish.
```

Done when (each group):

- [ ] It feels right on a release build on your slowest phone, ideally an older Android (the skill's rule: nothing else counts as verified)
- [ ] With Reduce Motion on, the same screens fade instead of moving

Step C.3 — Navigation and tab bar

```
Using animate-expo and apple-design, fix navigation motion: no animation on tab switches; native stack defaults for pushes; real form sheets (presentation formSheet) for the checkout, invite and report sheets instead of custom ones; and the iOS large-title header where a screen has one. Keep our custom floating tab bar, because native tabs can't show the raised gold Host button, but make the active dot move in under 150ms with no bounce, and the Host button open its sheet with the skill's sheet spring.
```

Done when:

- [ ] Switching tabs feels instant, and sheets drag down to close with momentum

Step C.4 — Add only the motion that earns it

```
Use find-animation-opportunities on Home, event detail, Tickets, the Prayer Wall and onboarding. Propose motion only for rare, meaningful moments (first RSVP, answered prayer, check-in, onboarding finish), and list what you rejected and why. Wait for my approval, then build the approved ones with animate-expo.
```

Step C.5 — Design polish pass

```
Use emil-design-eng and apple-design to review each main screen (Home,
Explore, event detail, checkout, Tickets, Community, Prayer Wall, Profile,
host dashboard, scanner): shadows versus borders, spacing rhythm, type
tracking and line height for DM Serif Display and Montserrat, frosted
materials, press states and contrast in both themes. Fix issues one screen
per commit, staying inside our design tokens and docs/DESIGN.pdf.
```

Step C.6 — Try to break it

```
Use break-ui on EventCard, PostCard, PrayerCard, ProfileHero,
WalletTicketCard, ChurchRow, ShiftCard and the host stat tiles: very long
names and church names, one-letter names, 10,000+ attendee counts, emoji,
Spanish and Korean text, missing photos and empty lists. Show each worst case
in /styleguide behind a Worst case toggle, then fix everything that breaks.
```

Done when:

- [ ] /styleguide's Worst case toggle shows nothing clipped, overlapping or unreadable

Optional: run /prototype on the event card and the ticket card to compare three variations each before you settle the final look.

Phase D: Test with real churches and launch (weeks 7–9)

Goal: real people use test builds, a developer checks the money and security code, and both stores approve the app.

Step D.1 — Accounts and production setup (you)

- [ ] Apple Developer organization account and Google Play Console (from the earlier plan's "Start now" list)
- [ ] Sentry DSN, Google Maps Android key and EAS credentials added to EAS secrets, not the repo
- [ ] Apply the Supabase migration to production after it has run cleanly on the dev project for a week
- [ ] Cancel or skip the Median plan; the native app replaces it

Step D.2 — Test release to real people

```
Set up EAS preview builds for iOS (TestFlight) and Android (Google Play
internal testing), walk me through submitting both, and create the demo
reviewer account and seeded events described in docs/STORE.md. Add a simple
in-app "Send feedback" link in Settings that emails support with the app
version and device.
```

- [ ] 2–3 churches host real events in the test build and 20–30 people RSVP and buy test tickets
- [ ] Watch Sentry and fix what testers report, one issue per session

Step D.3 — Developer review (paid, 4–6 hours)

Hire someone with React Native, Stripe Connect and Supabase experience to review payments, row-level security, account deletion and the Phase B server changes. Prepare with the prompt in docs/REVIEW.md, give read-only repo access and test keys only.

Step D.4 — Go live

```
Follow the launch checklist in docs/STORE.md: switch to live Stripe keys and
the live webhook in EAS secrets and Railway, confirm Supabase backups, build
production versions with eas build --profile production, and submit with eas
submit. Give me the checklist in order, including a $1 real ticket purchase I
make myself and then refund.
```

Done when:

- [ ] A lawyer has reviewed the legal drafts in content/legal.ts
- [ ] Your $1 live purchase and refund work
- [ ] Both stores approve the app

Sources

- ChristLink-native repository (read October 18, 2026: CLAUDE.md, docs/STATUS.md, docs/PROGRESS.md and the motion code)
- Emil Kowalski's skills repository
- Expo: App Store Connect minimum SDK 26
- Expo: Expo Go and the App Store in May 2026